



THE LONDON SCHOOL
OF ECONOMICS AND
POLITICAL SCIENCE ■

Week 1: The Computational Core

MY472: Data for Data Scientists

<https://lse-my472.github.io/>

Autumn Term 2026

Ryan Hübert

Associate Professor of Methodology

80/20 rule of data science:

80% data preparation, 20% data analysis

MY472 is about the 80%

Course philosophy

This course will teach you how to *prepare data* using Python

- ▷ Prepare = collect, wrangle, store, convert, etc.

MY472 is the “missing semester” of your data science education

- ▷ Not an introductory Python course (see MY470), and not a course on *data analysis*

This is a graduate course so we will move fast

- ▷ Lectures (Wednesdays) are conceptual; seminars (Thursdays) are hands-on—attend your assigned seminar every week
- ▷ Phones away, no headphones, laptops for course work only
- ▷ **Do not record or photograph an instructor or another student without their explicit consent**

Course outline

I. Your machine: how a computer holds data, and how to keep it safe

▷ W1 The computational core · W2 Digital data · W3 Digital security

II. Structured data: the “canonical” form, and how to show it

▷ W4 Tabular data · W5 Databases · W7 Visualisation

III. Data held elsewhere: getting data from someone else’s computer

▷ W8 APIs and remote computing · W9 Scraping webpages

IV. Unstructured data: imposing structure on signals

▷ W10 Audio and images · W11 Text as data

You and generative AI

Why learn these core skills when AI can do the work for you?

At work, you'll supervise AI agents

- ▷ You can't judge the quality of work you can't do yourself
- ▷ And an agent on your machine is a security risk if you can't follow what it's doing

At school, you learn the skills to be a good manager of AI agents, and learning is like going to the gym

- ▷ Nobody would ever hire someone to lift weights for them — so, don't hire AI to learn for you
- ▷ We're your personal trainers: we pick the exercises and make sure you do them properly

Computing requirements

A key starting point: flexibility with (and curiosity about) computing environments is a key requirement for data science

You will do coursework in the Python programming language

- ▷ Read website for more details on what this means

You must bring a laptop to seminar that can run required software

- ▷ Most laptops are fine, but no iPads, tablets or Chromebooks

Course materials (and tests) developed on Apple laptops

- ▷ Not a huge deal: you'll mostly be writing Python inside Positron
- ▷ During term, Windows users may have some extra set up steps

Summative assessments

Two in-person practical tests, closed-book, on LSE Macbooks:

1. **Mid-term (30%)**: 2 hour test during week 7 seminar
2. **Final (70%)**: 3 hrs on Friday 15 January 2027
 - Listed as a “lecture” in your Timetable; you will be randomly assigned one of the two time slots

Make sure you are available for both tests.

More details to come, but you will write Python code and answer conceptual questions

- ▷ Seminar exercises are designed to be good practice

If you have approved adjustments, contact me by week 2

Do this today (please)

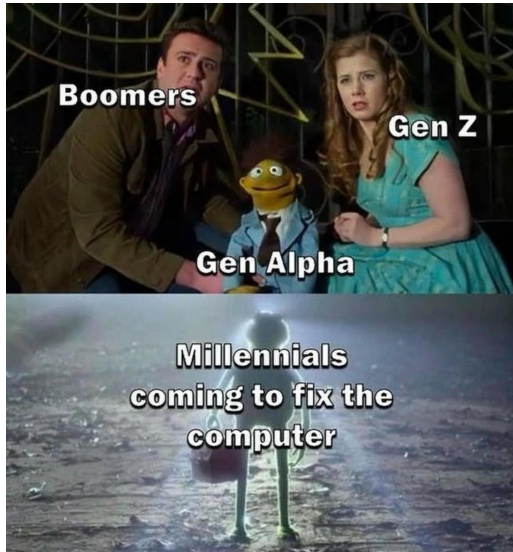
1. Carefully review the website: <https://lse-my472.github.io/>
2. Complete the set up steps:
<https://github.com/lse-my472/at26-getting-started>
 - GitHub username must be formatted LSE-XXXXX-2627 (candidate number **not** student ID)
3. Complete onboarding survey ASAP to get access to the GitHub organisation (not immediate)
4. When you get access, “Watch” the MY472 community GitHub repo at <https://github.com/lse-my472/at26-community>

Need help? GTA drop-in:

- ▷ Wed 30 Sep, 16:00–17:00 (CON.1.01)
- ▷ Wed 7 Oct, 16:00–17:00 (CBG.1.08)

Outline

- 1 Computational landscape
- 2 Programming for data science
- 3 Directory management in Python
- 4 Version control with Git



Directories and files

A **directory** is a location on a computer

- ▷ Directories are containers for computer **files**
- ▷ Often referred to as “folders” (hence the folder icons)
- ▷ A **working directory** contains files you’re currently using—it’s where you are currently working
- ▷ Sometimes tricky to know what your working directory is

Directories can be nested

- ▷ Often refer to the “top” directory as the **root directory**
- ▷ Directories inside of other directories are called **subdirectories** (or **subfolders**)
- ▷ A **parent directory** is one level up from a file or directory

Directories and files

Every directory on a computer has an “address” called a **(file) path**

- ▷ macOS example: `/Users/r.hubert/Documents/`
- ▷ Windows example: `C:\Users\HUBERTR\Documents\`

Note: on many machines, `~` is a shortcut for the “home” directory

- ▷ A home directory is a user’s root directory
- ▷ For me: `~/Documents/` $\Leftrightarrow$ `/Users/r.hubert/Documents/`

File paths can be **absolute** or **relative** to working directory

- ▷ `/Users/r.hubert/Documents/` is an absolute file path
- ▷ `Documents/` is a relative file path if my current working directory is `/Users/r.hubert/`

Directories and files

Files also have file paths like `/Users/r.hubert/Example.txt`

- ▶ Files always end with a **file extension** like `.txt` or `.py`, although some operating systems hide them by default
- ▶ In file paths, files always end with a file extension, directories will end with a slash or not: `~/Documents` ⇔ `~/Documents/`
- ▶ Some files are marked as **hidden files**, but data scientists usually need (and want) to see them
- ▶ Get in habit of showing extensions (`macOS/Windows`) and showing hidden files (`Windows`)
 - Show hidden files in macOS Finder: `Cmd+Shift+.`
- ▶ Copy absolute path of a file/folder with `Option+Right-click` (macOS) or `Shift+Right-click` (Windows)

Directories and files

There are many naming conventions you should learn/use

- ▷ Only use alphanumeric characters, underscores and hyphens
- ▷ Do not use spaces or other symbols
- ▷ Especially important: do not use slashes or periods!
- ▷ Use concise but descriptive names for folders and files:
 - Folders: `data`, `results`, `figures`, `code`
 - Files: `data_raw.csv`, `clean_data.py`, `data_clean.csv`

Get in the habit of building organised directory structures that others can understand and that will stand the test of time

- ▷ We'll evaluate you on this in MY472—use *good judgement*

Be intentional about where your data is!



The command line

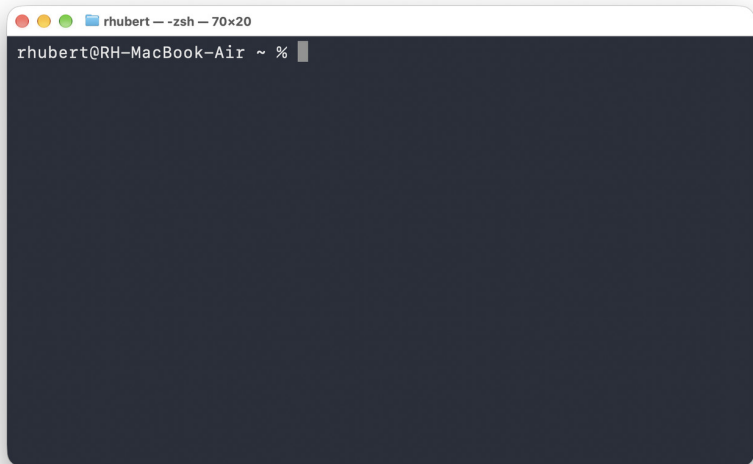
There are two primary ways users “talk to” their computers

- ▷ **Graphical user interface (GUI)**: “point-and-click” interface—this is how most users interact with computers
- ▷ **Command line interface (CLI)**: text-based interface where a user types instructions to computer via a **command line**
 - Mac/Linux: use command line in the **Terminal** app
 - Windows: use command line in **PowerShell**

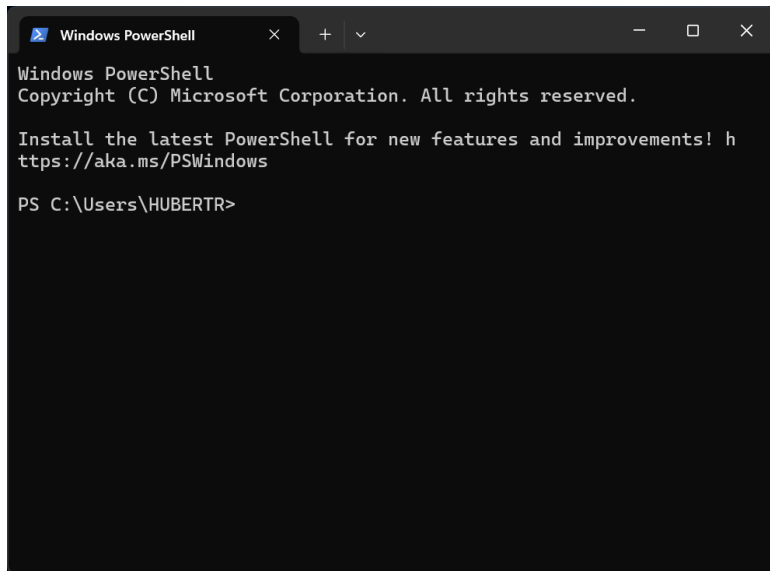
Can do basic computer functions in both, such as moving files around, editing text files, etc.

Many apps have a GUI *and* a CLI, but some only have one

The command line



The command line

A screenshot of a Windows PowerShell terminal window. The window has a dark gray title bar with the text "Windows PowerShell" and standard window control buttons (minimize, maximize, close). The terminal content is as follows:

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! h
ttps://aka.ms/PSWindows

PS C:\Users\HUBERTR>
```

The command line

The command line is always in a particular directory

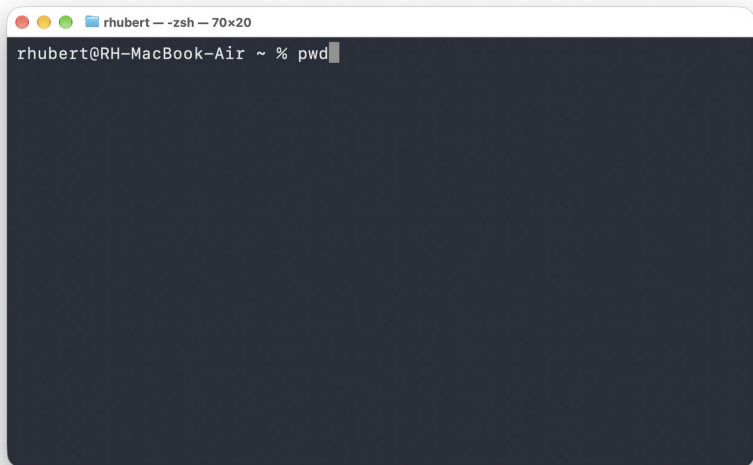
- ▷ It usually launches inside your home directory

You'll often need to navigate to other directories to do stuff

Some useful commands (Linux/Unix/macOS/PowerShell):

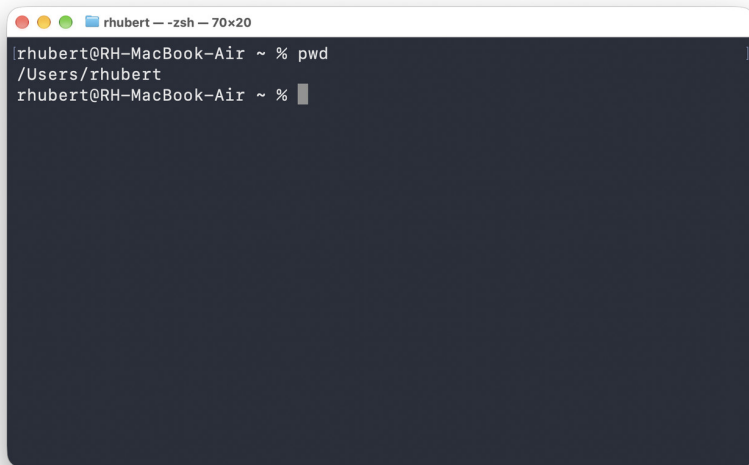
- ▷ `pwd`: print working directory
- ▷ `ls`: list folders and files in the working directory
- ▷ `cd <path>`: change directory using absolute/relative filepaths
 - `cd ..` goes up one folder level
- ▷ `mkdir <path>`: make a new directory

Directories with the command line

A terminal window with a dark blue background and a white title bar. The title bar contains three colored window control buttons (red, yellow, green) and a folder icon followed by the text "rhubert -- zsh -- 70x20". The terminal content shows the prompt "rhubert@RH-MacBook-Air ~ % " followed by the command "pwd" and a cursor. The rest of the terminal area is empty.

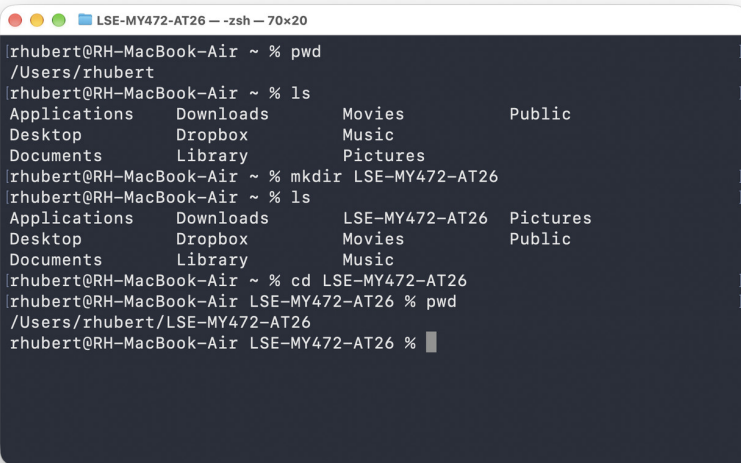
```
rhubert@RH-MacBook-Air ~ % pwd
```

Directories with the command line

A terminal window with a dark blue background and a white title bar. The title bar contains three colored window control buttons (red, yellow, green) and the text 'rhubert -- zsh -- 70x20'. The terminal shows a prompt 'rhubert@RH-MacBook-Air ~ %' followed by the command 'pwd'. The output of the command is '/Users/rhubert'. The prompt is followed by a cursor.

```
rhubert@RH-MacBook-Air ~ % pwd  
/Users/rhubert  
rhubert@RH-MacBook-Air ~ %
```

Directories with the command line

A terminal window titled "LSE-MY472-AT26 -- -zsh -- 70x20" is shown. It displays a series of commands and their outputs. The user starts in the home directory, lists the contents, creates a new directory named "LSE-MY472-AT26", lists the contents again to confirm its creation, and then navigates into the new directory.

```
LSE-MY472-AT26 -- -zsh -- 70x20
[rhubert@RH-MacBook-Air ~ % pwd
/Users/rhubert
[rhubert@RH-MacBook-Air ~ % ls
Applications      Downloads         Movies            Public
Desktop           Dropbox          Music
Documents         Library          Pictures
[rhubert@RH-MacBook-Air ~ % mkdir LSE-MY472-AT26
[rhubert@RH-MacBook-Air ~ % ls
Applications      Downloads         LSE-MY472-AT26   Pictures
Desktop           Dropbox          Movies            Public
Documents         Library          Music
[rhubert@RH-MacBook-Air ~ % cd LSE-MY472-AT26
[rhubert@RH-MacBook-Air LSE-MY472-AT26 % pwd
/Users/rhubert/LSE-MY472-AT26
rhubert@RH-MacBook-Air LSE-MY472-AT26 %
```

Creating and deleting files with the command line

You can create, modify and preview files with:

- ▷ `touch`: create a (blank) file or update time metadata
- ▷ `echo` (with `>`): write text to a new text file
- ▷ `cat`: print the contents of a text file
- ▷ `mv <path1> <path2>`: move path1 to path2
- ▷ `cp <path1> <path2>`: copy path1 to path2

You can *immediately** remove (delete) files and folders with:

- ▷ `rmdir`: remove (delete) a directory – only if empty!
- ▷ `rm`: remove (delete) a file
 - If you want to delete a non-empty folder, `rm -r`

Note: any deletion on the command line is immediate

Creating and deleting files with the command line

```
lec01 — -zsh — 70x20
[rhubert@RH-MacBook-Air ~ % cd LSE-MY472-AT26
[rhubert@RH-MacBook-Air LSE-MY472-AT26 % echo "LSE" > myfile.txt
[rhubert@RH-MacBook-Air LSE-MY472-AT26 % ls
myfile.txt
[rhubert@RH-MacBook-Air LSE-MY472-AT26 % cat myfile.txt
LSE
[rhubert@RH-MacBook-Air LSE-MY472-AT26 % echo "MY472" >> myfile.txt
[rhubert@RH-MacBook-Air LSE-MY472-AT26 % cat myfile.txt
LSE
MY472
[rhubert@RH-MacBook-Air LSE-MY472-AT26 % rm myfile.txt
[rhubert@RH-MacBook-Air LSE-MY472-AT26 % ls
[rhubert@RH-MacBook-Air LSE-MY472-AT26 % mkdir lec01 && echo "LSE MY472"
" > lec01/myfile.txt
[rhubert@RH-MacBook-Air LSE-MY472-AT26 % ls
lec01
[rhubert@RH-MacBook-Air LSE-MY472-AT26 % cd lec01 && cat myfile.txt
LSE MY472
[rhubert@RH-MacBook-Air lec01 %
```

Operating system types

We won't go deep into operating systems, but most are **Unix-like**

- ▷ macOS, iOS, Linux, Android, ChromeOS, and most servers
- ▷ Built around the command line and small, combinable tools
- ▷ Most modern programming tools are built for Unix first

The big exception is **Windows**

- ▷ Built around a graphical interface first, GUI-forward
- ▷ Can run modern programming tools, but often needs extra setup or compatibility tools

Data scientists must be comfortable with Unix-like systems

Outline

- 1 Computational landscape
- 2 Programming for data science**
- 3 Directory management in Python
- 4 Version control with Git

Programming languages: overview

A **computer program** is a sequence of instructions that a computer is asked to perform

Computer programs are written in **programming languages** that control how the computer interprets instructions

Programming languages can be *low-level* (e.g., C++) or *high-level* (e.g., R or Python)

Data scientists typically work with high-level languages that are built for **object oriented programming**

- ▷ OOP organises code around **objects** that are defined in code

Python and R

Python and **R**: two high-level OOP languages, free and open-source, with large ecosystems of useful tools in “packages”

In MY472: we use Python (this is new from previous years)

- ▷ It's more popular among data scientists
- ▷ It's more “general purpose” and thus stronger for working with your computer: files, networks, other programs

R targets statistical computing: it's better out of the box for statistics and plotting, clunkier for working with your computer

As a data scientist you'll probably need to be bilingual

- ▷ But if you learn one, you can learn the other quickly

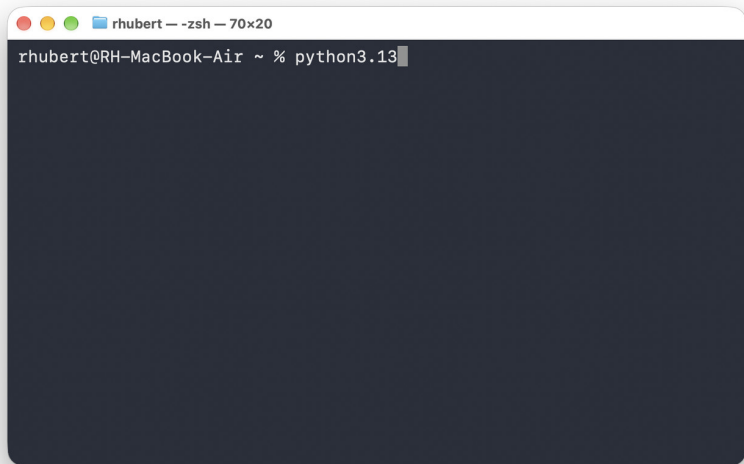
Coding

You can write lines of Python code directly into a **console** and get an immediate response

- ▷ When you ask the console to process your code, you are **running** or **executing** your code
- ▷ The response is printed in the console and it's usually referred to as the **output**

A Python console can be launched from inside many applications on a computer, the simplest probably being the Terminal

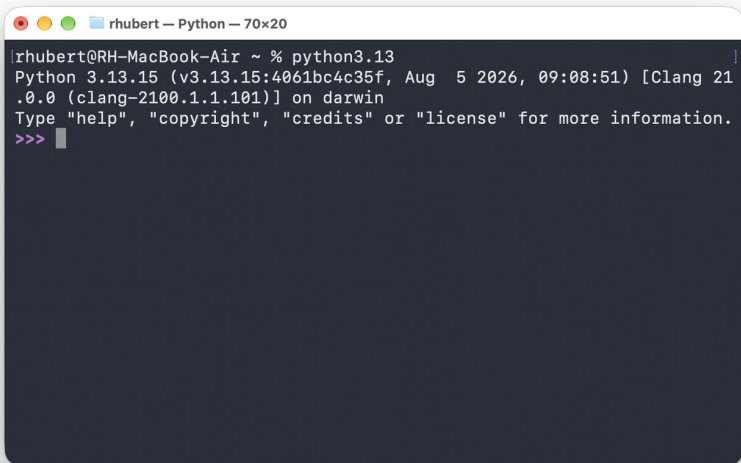
Coding



A terminal window with a title bar containing three colored circles (red, yellow, green) and a folder icon, followed by the text "rhubert — -zsh — 70x20". The terminal area has a dark background and displays the prompt "rhubert@RH-MacBook-Air ~ %" followed by the command "python3.13" and a cursor.

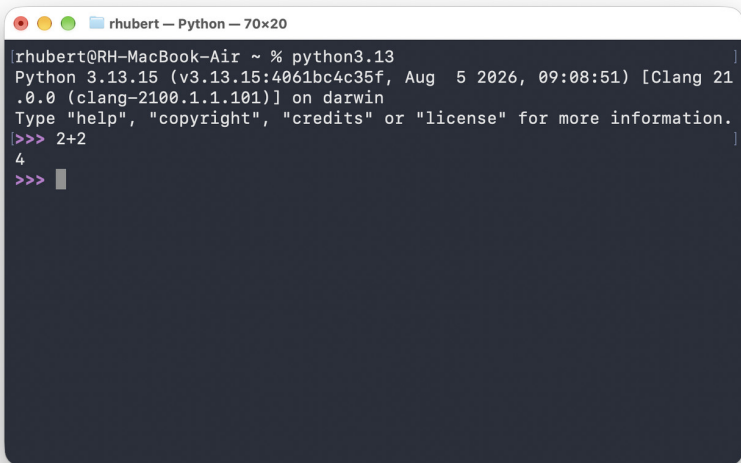
```
rhubert@RH-MacBook-Air ~ % python3.13
```

Coding

A terminal window titled "rhubert — Python — 70x20" with standard macOS window controls. The terminal shows the command "python3.13" being executed, followed by the Python version and build information: "Python 3.13.15 (v3.13.15:4061bc4c35f, Aug 5 2026, 09:08:51) [Clang 21.0.0 (clang-2100.1.1.101)] on darwin". It then displays the prompt "Type 'help', 'copyright', 'credits' or 'license' for more information." and the interactive prompt ">>>" with a cursor.

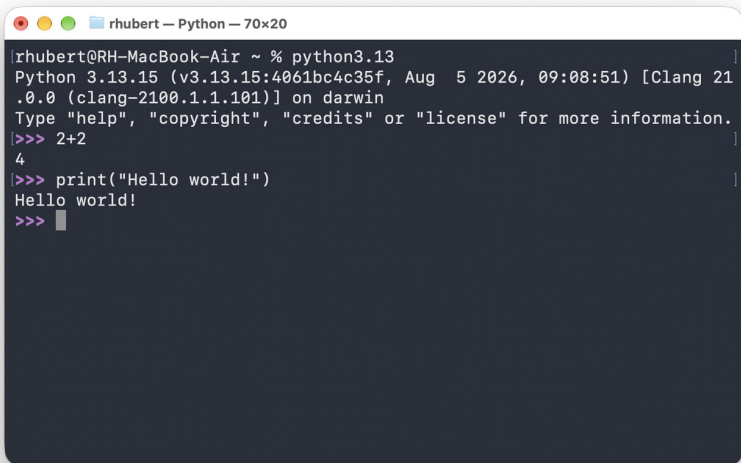
```
rhubert@RH-MacBook-Air ~ % python3.13
Python 3.13.15 (v3.13.15:4061bc4c35f, Aug 5 2026, 09:08:51) [Clang 21
.0.0 (clang-2100.1.1.101)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>> █
```

Coding



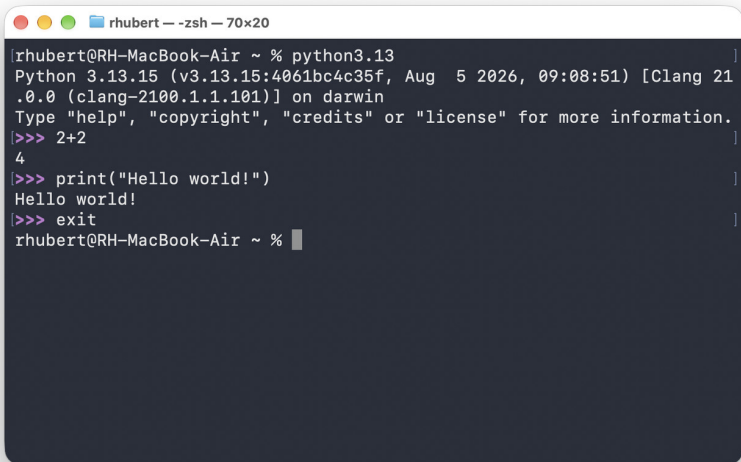
```
rhubert — Python — 70x20
[rhubert@RH-MacBook-Air ~ % python3.13
Python 3.13.15 (v3.13.15:4061bc4c35f, Aug  5 2026, 09:08:51) [Clang 21
.0.0 (clang-2100.1.1.101)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
[>>> 2+2
4
>>> ]
```

Coding



```
rhubert — Python — 70x20
[rhubert@RH-MacBook-Air ~ % python3.13
Python 3.13.15 (v3.13.15:4061bc4c35f, Aug  5 2026, 09:08:51) [Clang 21
.0.0 (clang-2100.1.1.101)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
[>>> 2+2
4
[>>> print("Hello world!")
Hello world!
>>> █
```

Coding



```
rhubert — -zsh — 70x20

[rhubert@RH-MacBook-Air ~ % python3.13
Python 3.13.15 (v3.13.15:4061bc4c35f, Aug  5 2026, 09:08:51) [Clang 21
.0.0 (clang-2100.1.1.101)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
[>>> 2+2
4
[>>> print("Hello world!")
Hello world!
[>>> exit
rhubert@RH-MacBook-Air ~ %
```

Coding

You can also write lines of code into a plain text file, which you can execute whenever you choose

- ▷ You can run lines of code as you are drafting
- ▷ You can include **comments**, which are lines in a script that will not be interpreted as instructions by the console
- ▷ In Python, comments are preceded by a hash: `#`
- ▷ Can “comment out” code that is broken or you no longer want to run (but that you want to keep for posterity)

You can write code in a text editor (e.g. Notepad or TextEdit).

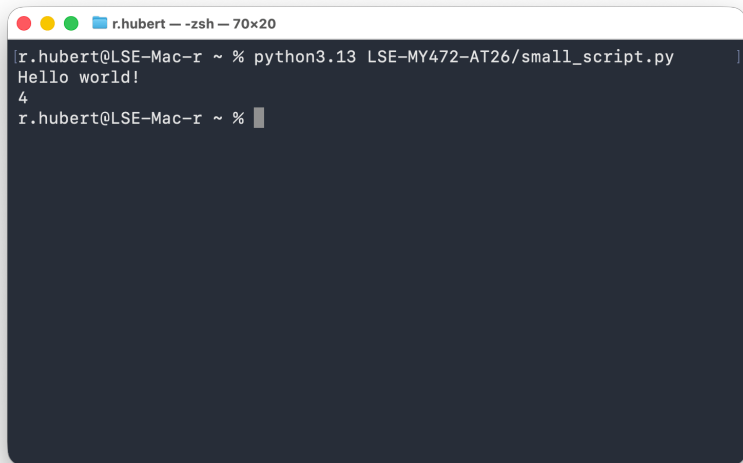
- ▷ But then you need a console to run the code
- ▷ For example, you can again use the command line

Coding

 small_script.py

```
2+2                                # This won't print
print("Hello world!")             # This will print
print(2+2)                        # This will print
# print("LSE")                    # This won't print
```

Coding



A terminal window titled "r.hubert - zsh - 70x20" with standard macOS window controls (red, yellow, green buttons). The terminal shows a command to run a Python script, the output "Hello world!", and a prompt for the next command.

```
r.hubert@LSE-Mac-r ~ % python3.13 LSE-MY472-AT26/small_script.py  
Hello world!  
4  
r.hubert@LSE-Mac-r ~ %
```

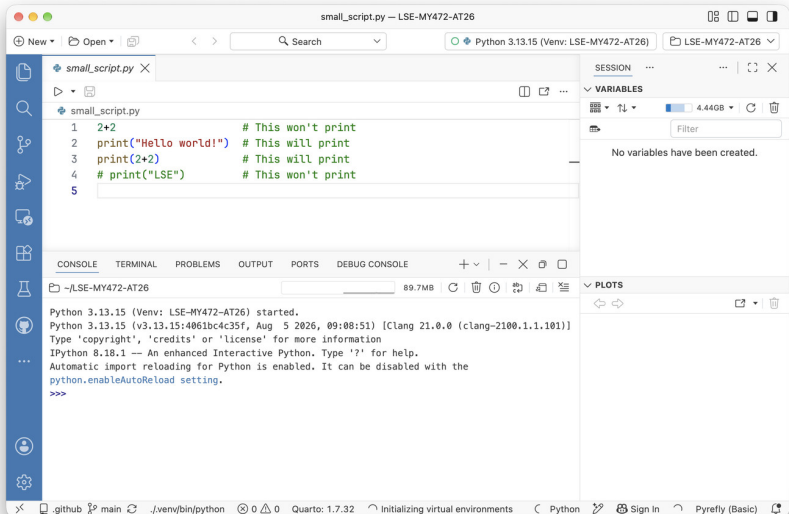
Integrated development environments (IDEs)

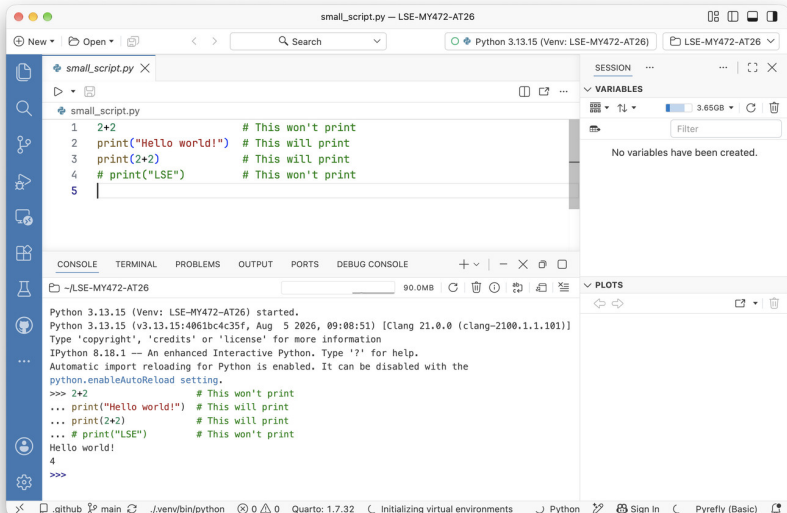
Now more common to use an **integrated development environment (IDE)** instead of text file + command line

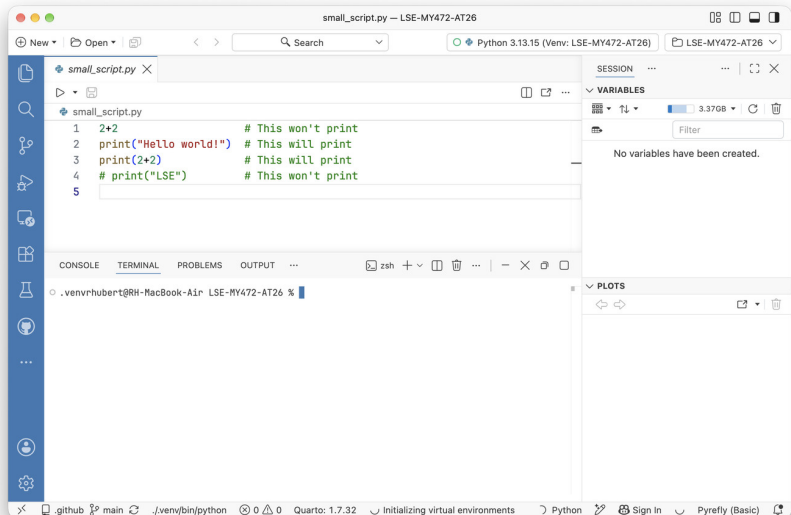
- ▷ Provides a suite of tools for developing and running code
- ▷ Some examples: [Positron](#), [VS Code](#), [PyCharm](#), [RStudio](#)

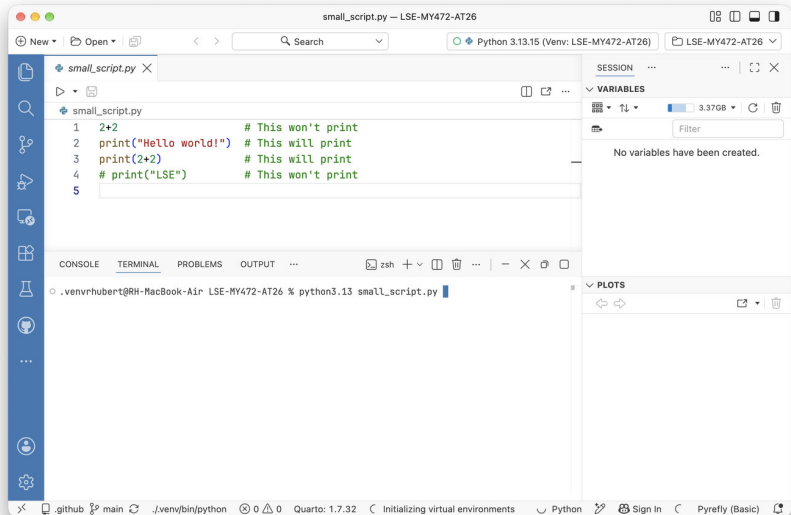
In this class, we will teach you using Positron

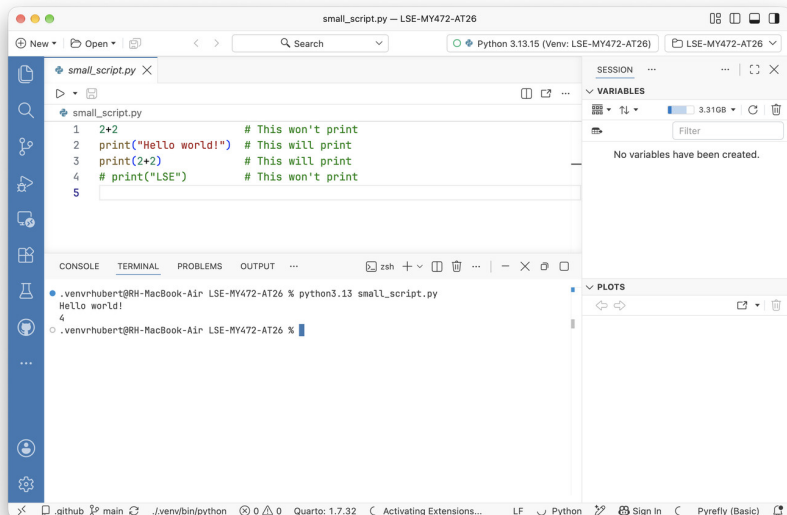
- ▷ We use it because: (1) it's built for both Python and R, and (2) you use VS Code in MY470 (and other courses)
- ▷ It is a “fork” of VS Code with customisations for data science, so it looks (and works) very similarly to VS Code
- ▷ Feel free to use VS Code if you prefer











Executing Python: packages

[AI-DRAFTED]

A **package** (sometimes called a **library**) is a bundle of files (mostly `.py` scripts) that adds new functionality

- ▷ Offers pre-built tools (mostly Python **functions** and **classes**) that you use without having to write yourself
- ▷ A single `.py` file in a package is called a **module**

Use objects in a package by importing it first:

```
import pandas
pandas.read_csv("/Users/rhubert/Documents/sample.csv")
```

The **Python standard library** is a set of packages included with Python when you install it, e.g. `os`, `shutil`, `re`, etc.

Most other Python packages are shared through **PyPI** (e.g. `pandas`) and must be installed — but installed *where*?

Python packages install *somewhere* on your computer

- ▷ Loosely refer to this location as the Python **environment**
- ▷ You have a default environment (a “system build”)
- ▷ Using the system build for different projects can cause problems, e.g.: Project A needs **pandas** version 2.2; an older Project B breaks on anything after **pandas** version 1.5

A **virtual environment** (or **venv**) is a self-contained folder holding its own copy of Python and its own set of packages

- ▷ Good data science practice: one venv per project
- ▷ It is just a directory, often called **.venv/** inside your project folder (here, a hidden folder)
- ▷ You **activate** a venv to tell your computer “use this one” — Positron can do this for you (see course set up)

Data science results depend on code, data, *and* software

- ▷ Package updates change defaults, rename functions, and occasionally change results silently

A venv plus a **requirements file** (`requirements.txt`) listing exact package versions lets anyone rebuild the same environment

- ▷ *Across runs*: analysis works after laptop updates
- ▷ *Across machines*: co-authors, colleagues and others
- ▷ Many journals expect replication packages specifying versions
- ▷ Caution: fixes packages, not operating system!

In MY472: we treat the course as a project, and use one venv built from a `requirements.txt` we provide (and update weekly)

Writing Python: scripts versus notebooks

With Python (and other languages) there are actually two kinds of files you can write code into:

- ▷ A **script** contains only lines of code and comments (again, preceded by a #)
 - With the Python language, scripts are saved in `.py` files
- ▷ A **notebook** contains both written text plus code, where the code is contained within demarcated **chunks**
 - The written text can be formatted using **markdown**
 - You can still run lines of code from within chunks
 - You can also **compile** (or **knit** or **render** or **preview**) the notebook to a nicely formatted `.html` or `.pdf` file
 - The compiled document will contain formatted text, chunks of code, and the code output

Writing Python: Python notebooks

Most common kind of Python notebook is the **Jupyter notebook**

- ▷ *Not* plain text—interactive documents, complex file structure

A plain text alternative: **Quarto notebooks**

- ▷ Quarto notebooks are saved as `.qmd` files
- ▷ Very useful: simple plain text files that can be used with many programming languages and for many purposes

In this course, we will use `.qmd` files

- ▷ We will write our code (and text) in `.qmd` files and then compile the end product to nicely formatted `.html` files
- ▷ For assessments, you will need to submit `.qmd` and `.html` files
- ▷ If you are familiar with R Markdown, Quarto is very similar

Outline

- 1 Computational landscape
- 2 Programming for data science
- 3 Directory management in Python**
- 4 Version control with Git

Directory management in Python

Just like the command line, Python always starts in a directory

```
import os  
print(os.getcwd())
```

```
/Users/r.hubert/Library/CloudStorage/Dropbox/Teaching/MY472-AT26/week01
```

By far the most common error in MY472:

```
with open("myfile.txt", mode = "r") as f:  
    print(f.read())
```

```
FileNotFoundError: [Errno 2] No such file or directory: 'myfile.txt'
```

File `myfile.txt` isn't in Python's (current) working directory:

```
os.listdir(os.getcwd())
```

```
['my472-w01-computation-core.qmd', 'my472-w01-computation-core.pdf']
```

Sidebar on error handling

Huge part of data science (& coding): dealing with errors in code

Computers are very literal—they cannot interpret subtext

- ▷ Your *intended* instructions might not be what computer understands your instructions to be
- ▷ (One reason that “vibecoding” can be quite dangerous!)

More than 95% of errors in MY472 are human errors

A key skill: learning how to read and interpret error messages

- ▷ **I want to emphasise: please read error messages!**
- ▷ Learn to love [Stack Overflow](#), help docs & AI assistants

Directory management in Python

How to deal with this?

1. Try to change the working directory: `os.chdir("<path>")`
2. Specify the *full* path when opening files

In MY472, we'll do #2:

```
wdir = os.path.expanduser("~/LSE-MY472-AT26/lec01")  
print(wdir)
```

```
/Users/r.hubert/LSE-MY472-AT26/lec01
```

```
path_to_file = os.path.join(wdir, "myfile.txt")  
print(path_to_file)
```

```
/Users/r.hubert/LSE-MY472-AT26/lec01/myfile.txt
```

```
with open(path_to_file, mode = "r") as f:  
    print(f.read())
```

Directory management in Python

You can also check if directories exist, and make them if not

```
new_folder = os.path.expanduser("~/TestFolder")  
print(new_folder)
```

```
/Users/r.hubert/TestFolder
```

```
os.path.exists(new_folder)
```

```
False
```

```
if not os.path.exists(new_folder):  
    os.mkdir(new_folder)  
os.path.exists(new_folder)
```

```
True
```

In general: make an object with the folder or file name to avoid copy/paste errors in subsequent lines

Local versus remote

So far, we just saw examples of **local** directories

- ▷ “Local” just means *on your computer*

Files can also be stored in **remote** directories

- ▷ “Remote” means *on another computer* (a **server**)

To use a file from a remote directory on your own computer, you have to **download** it first

Python’s standard library `urllib.request` has tools for downloading files using URLs

Local versus remote

```
from urllib.request import urlretrieve

remote_file = "https://lse-my472.github.io/README.md"
local_file = os.path.join(wdir, "README.md")

urlretrieve(remote_file, local_file)

with open(local_file, "r") as f:
    raw_text = f.read()

print(raw_text)
```

LSE MY472: Data for Data Scientists

This is the repository for the LSE MY472 course website.

Outline

- 1 Computational landscape
- 2 Programming for data science
- 3 Directory management in Python
- 4 Version control with Git

Version control

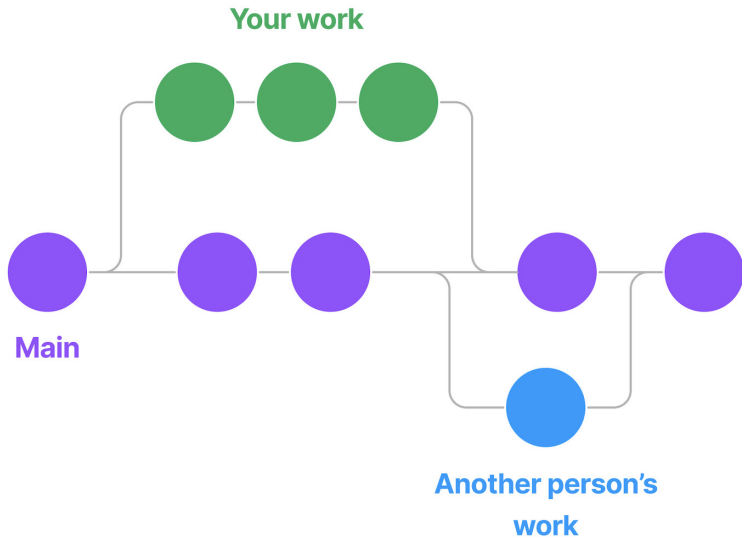
A **version control system (VCS)** is key when working on code, particularly when collaborating

It keeps records of changes in files: *who* made *which* changes *when*

Allows for possibility of reverting changes and going back to previous states

When a VCS keeps the entire code and history on each collaborator's computer, it is called **distributed**

The main idea(s)



Git and GitHub

Git is a very popular distributed version control system

- ▷ It's software you install on your computer, which tracks files

A **repository** (or **repo**) is a collection of files (code, data, etc.) that are tracked by Git

- ▷ A repo is stored in a directory (recall: folder) on your computer with some special (hidden) files in them
- ▷ You can instruct Git to ignore and not track some files

GitHub is a web-based service that hosts collections of code online (with many extra functionalities)

- ▷ GitHub is a cloud backup/distribution system for local repos

Git repositories

The screenshot shows a GitHub repository page for 'political-appointments-hubert-copus' by user 'ryanhubert'. The repository is public and has 12 commits, 1 branch, and 0 tags. The file list includes various R scripts, Python files, PDFs, and a README. The README is selected, showing the title 'Political Appointments and Outcomes in Federal District Courts' and a description of the repository's purpose.

Platform Solutions Resources Open Source Enterprise Pricing Search or jump to... Sign in Sign up

ryanhubert / political-appointments-hubert-copus Public

Code Issues Pull requests Actions Projects Security Insights

master 1 Branch 0 Tags Go to file Code

File	Commit	Time
Analysis	Initial commit	4 years ago
01_Preprocess.R	Add additional information about replication files and arch...	5 years ago
02_Predictions.py	Add additional information about replication files and arch...	5 years ago
03_Analysis_Appeals.py	Add additional information about replication files and arch...	5 years ago
04_Analysis_Main.R	Add additional information about replication files and arch...	5 years ago
05_Outputs.R	Fix typos in comments	4 years ago
Appeals Codebook 1971-2007.pdf	Initial commit	5 years ago
Appeals Codebook 2008 Forward_0.pdf	Initial commit	5 years ago
Civil Codebook 1988 Forward.pdf	Initial commit	5 years ago
LICENSE	Initial commit	5 years ago
README.md	Add more info	4 years ago
codebook-appeals.md	Initial commit	5 years ago
codebook-main.md	Initial commit	5 years ago
requirements.txt	Initial commit	5 years ago

ryanhubert Add more info 23c442a · 4 years ago 12 Commits

About
No description, website, or topics provided.

Readme MIT license Activity 0 stars 1 watching 1 fork Report repository

Releases
No releases published

Packages
No packages published

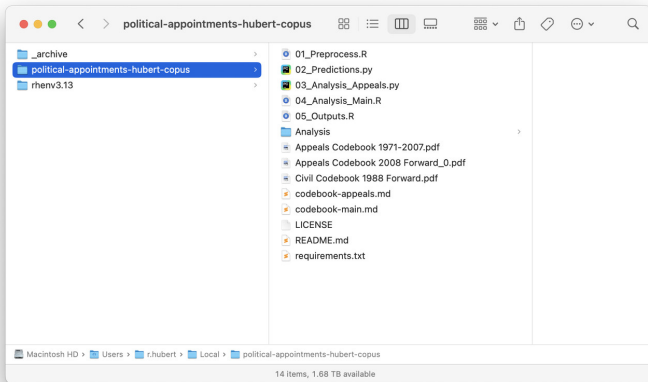
Languages
R 67.7% Python 32.3%

README MIT license

Political Appointments and Outcomes in Federal District Courts

This repository contains replication materials for "Political Appointments and Outcomes in Federal District Courts" by Ryan Hubert and Ryan Copus, which is forthcoming in the *Journal of Politics*.

Git repositories



Git workflow basics

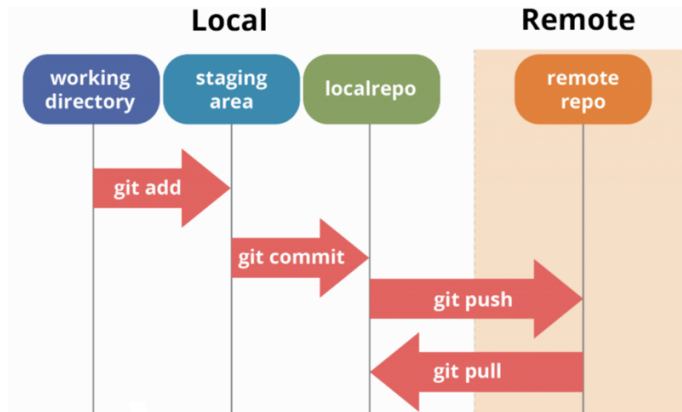
Most common way to work with Git on your computer (“locally”) is via the command line

- ▷ You *can* use the GitHub app or Git/GitHub integration in your IDE, but we will teach you on command line

These are the main commands we will use:

1. `git clone`: save a repo from a remote server to a local folder
2. `git pull`: pull files from remote to local
3. `git status`: check the status of the local repo
4. `git add`: stage files, preparing them for commit
5. `git commit`: commit changes to the local repo (always need to add a message)
6. `git push`: push changes from local to remote

Git workflow basics



Adapted from: Molly Nemerever, [Git](#), [GitHub](#), & [Workflow Fundamentals](#)

Advanced usage of Git

In most real-world settings, you will need to know:

- ▷ **Fork**: make your own version of a repository
 - Note: you're *forking* off, not *cloning*
- ▷ **Branch**: a parallel version of a repo originating at a particular point
- ▷ **Merge**: combining branches together to maintain single "codebase"
- ▷ **Pull request**: GitHub based request to merge a branch or a fork into other code

You won't use these in this course.

- ▷ For practice: <https://learngitbranching.js.org/>